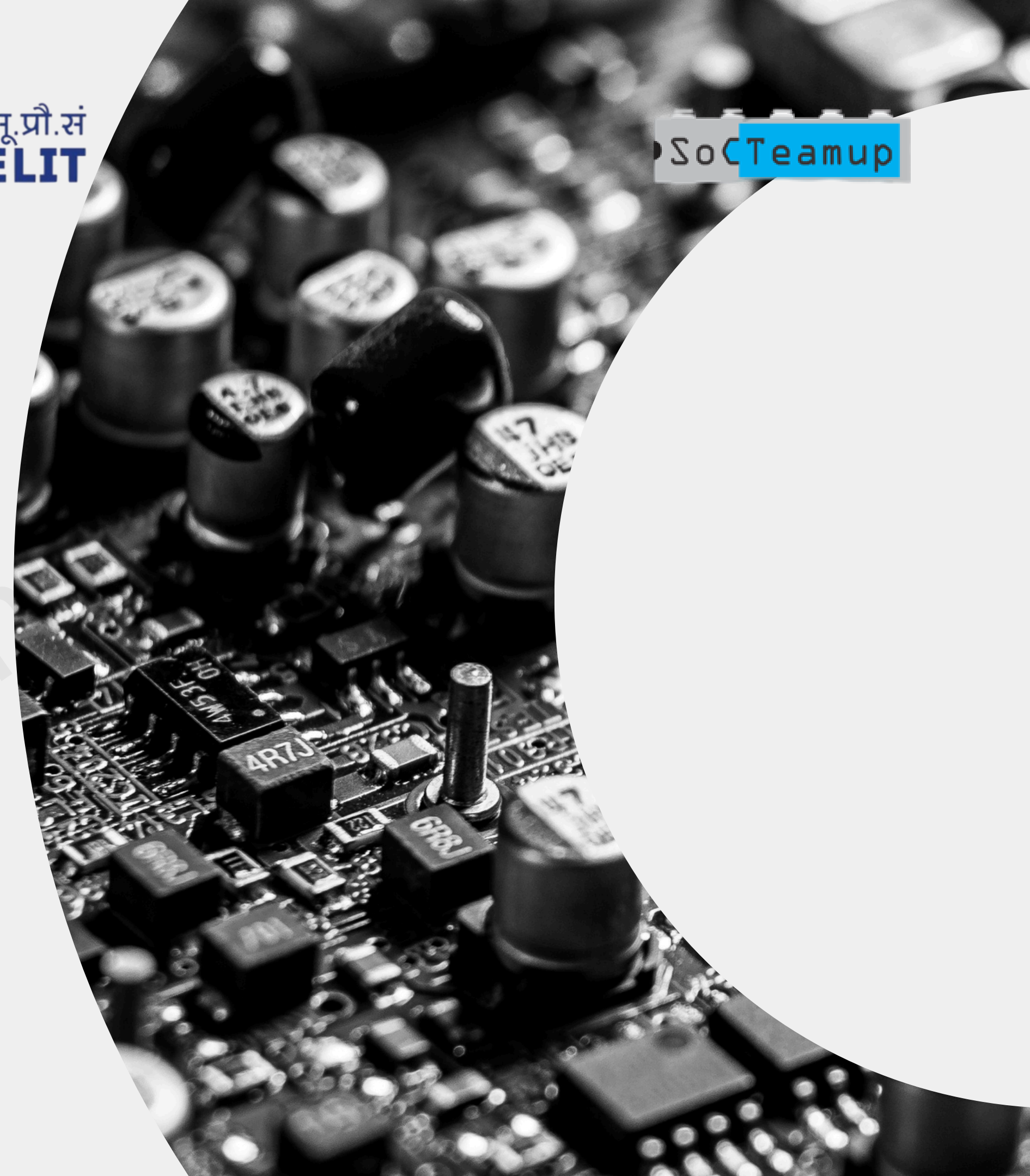


Area, Timing, Power trade-offs during **synthesis**





Outline

Importance of Trade-offs in RTL Synthesis
Defining Area, Timing, and Power
Role of Synthesis in Optimization
Area Optimization Strategies
Timing Optimization Techniques
Critical Path and Setup/Hold Concepts
Coding Style Impact on Timing
Power Optimization Techniques
Clock Gating Strategies
Trade-off Examples (Area vs Timing, Timing vs Power)
RTL Coding Example Affecting All Three Metrics
Tool-Specific Optimization in Yosys
Summary of Trade-offs

Importance of Trade-offs in RTL **Synthesis**

In real-world chip design:

- Perfect timing, lowest power, and smallest area cannot be achieved together.
- Design teams must find the right balance based on:
- Target frequency
- Power budget
- Area constraints (die cost, packaging)

RTL synthesis is where many of these decisions begin.

➔ **Definitions — Area, Timing, Power**

Area:

Area measures total silicon real estate used by gates, wires, and standard cells, typically in μm^2 or gate equivalents.

➔ **Definitions — Area, Timing, Power**

Timing:

Timing determines maximum achievable clock frequency through critical path analysis, measured by worst negative slack (WNS) and total negative slack (TNS). Paths must complete within clock period (setup) while data remains stable (hold).

➔ **Definitions — Area, Timing, Power**

Power:

Power splits into dynamic
and static power.

Dynamic scales with
frequency and toggle
rate; static worsens at
smaller nodes

➔ Role of Synthesis in Optimization

Synthesis converts RTL to optimized gate-level netlist targeting PPA goals.

Key Roles:

- Transforms high-level RTL into technology-specific gates
- Balances area, timing, power via iterative optimization passes
- Prepares design for physical implementation (place & route)

- Area Optimization
- Timing Optimization
- Power Optimization

Area Optimization

- Removes redundant logic and dead code
- Merges equivalent gates and shares resources
- Maps to smallest library cells possible

Role of Synthesis in Optimization

Timing Optimization

- Analyzes critical paths and inserts buffers
- Upsizes gates on slow paths
- Retimes registers to balance pipeline

Power Optimization

- Inserts clock gating automatically
- Selects high-Vt (low-leakage) cells
 - Reduces switching activity via logic restructuring,
Clock gating and Input masking

➔ Critical Path & Timing Constraints

Critical Path:

- Longest timing path from one flip-flop to another

Setup time:

- Minimum time data must be stable before clock edge

Hold time:

- Maximum Time data must remain stable after clock edge

- Violating these setup and hold time leads to functional errors or race conditions

➔ Coding Style Impact on Timing

Bad practices:

- Deeply nested if-else or case statements
- Complex arithmetic operations in a single line
- Accidental latches (incomplete if-else or case)

Good practices:

- Use registered outputs
- Split large expressions
- Apply hierarchy to isolate delays

Clock Gating Strategies

- Disable clock for inactive modules
- Synthesis tools recognize enable-based logic
- Use library clock gating cells (if available)

```
always @(posedge clk)
begin
if (enable)
q <= d;
end
```

- Avoid manually gating clocks using clk & enable

Trade-off Examples

Area vs Timing:

- Sharing a multiplex → saves area but increases delay

Timing vs Power:

- High-drive cells → fix setup slack but increase switching power

Area vs Power:

- Reusing registers saves area but increases switching activity if not gated

➔ RTL Coding Example

This affects:

- **Area:** Two arithmetic operations
- **Timing:** Conditional path selection adds delay
- **Power:** More logic → more toggling □ more power consumption

Optimize by using a shared ALU with a control signal

```
assign out = (sel) ? (a + b) : (a - b);
```

→ Tool-Specific Optimizations in Yosys

synth: General synthesis

opt_clean: Remove unused logic

stat: Reports gate count (proxy for area)

abc: Maps logic to library cells with timing
optimization

techmap: Apply technology mapping

- Use reports to compare results after each optimization pass

➔ Best Practices and Summary

Summary:

- Synthesis is a trade-off process among area, timing, and power
- Optimization depends on constraints, coding, and tool settings
- Early RTL decisions significantly affect post-synthesis outcome

Best Practices:

- Avoid deep logic chains → improve timing
- Reuse logic carefully → reduce area
- Use enables or gating → lower power
- Review synthesis reports regularly and iterate early

Thank You

echiphu

